

FloodGuard: AI-Powered Flood Risk Prediction System

Project Description

FloodGuard is a machine-learning-powered web application that predicts flood risk from weather and rainfall data. Given ten inputs (temperature, humidity, cloud cover, annual rainfall, and seasonal rainfall splits), the system returns an instant flood/no-flood classification along with a probability breakdown, helping residents and disaster-management teams act on data instead of guesswork.

The platform includes a training pipeline that compares four candidate classifiers (Decision Tree, Random Forest, KNN, and XGBoost) and automatically keeps the best performer; a Flask REST API that serves live predictions; a responsive web interface with dynamic, no-reload results; and a prediction-history log backed by a relational database.

Built with Flask and scikit-learn/XGBoost, and deployed publicly via Render, FloodGuard ensures a seamless experience from data entry to risk assessment, with clean separation between the ML pipeline, backend API, and frontend for easy maintenance and extension.

Scenarios

Scenario 1: A disaster-management officer enters this week's rainfall and humidity readings for their district. FloodGuard returns a 91% flood-risk probability, prompting an early warning to be issued to residents before conditions worsen.

Scenario 2: A resident checks conditions before a monsoon weekend. Cloud cover and rainfall figures produce a 12% flood probability, and the app shows a clear "Safe" badge, reassuring them there is no need to relocate belongings.

Scenario 3: A student studying the project reviews the /history page and sees a spread of past predictions with varying probabilities, illustrating how the model's confidence changes with different combinations of rainfall and humidity.

Technical Architecture

FloodGuard uses a three-tier architecture. The presentation layer is a responsive HTML/CSS/JavaScript frontend served through Flask's Jinja2 templates. The application layer is a Flask app-factory package (app/) that separates routing (routes.py), database models (models.py), and ML inference (ml.py). The data layer is a SQLite database (via Flask-SQLAlchemy) storing users, weather submissions, registered ML models, and prediction results, alongside the persisted model artifacts (flood_model.joblib, scaler.joblib, metadata.json) produced by the offline training pipeline (train.py).

Note: Since this project uses a database, its four tables (User, Weather_Data, ML_Model, Prediction_Result) form a simple ER diagram: User (1:N) Weather_Data (1:1) Prediction_Result (N:1) ML_Model. (Insert the visual ER diagram here.)

Pre-requisites

- Flask Framework Knowledge: <https://flask.palletsprojects.com/>
- scikit-learn / XGBoost Familiarity: <https://scikit-learn.org/>, <https://xgboost.readthedocs.io/>
- HTML, CSS, and JavaScript Skills: <https://developer.mozilla.org/>
- Python Programming Proficiency: <https://docs.python.org/3/>
- Version Control with Git: <https://git-scm.com/doc>
- Render Deployment Basics: <https://render.com/docs>

Project Workflow

Milestone 1: Data Preparation and Model Selection

- Activity 1.1: Source and load the flood dataset (flood_dataset.xlsx, 115 records).
- Activity 1.2: Perform exploratory data analysis — class balance and correlation heatmap.
- Activity 1.3: Clean the data (median imputation, IQR outlier clipping) and scale features.
- Activity 1.4: Train and compare Decision Tree, Random Forest, KNN, and XGBoost.

Milestone 2: Core Backend Development

- Activity 2.1: Design the Flask app-factory package (app/__init__.py) and SQLAlchemy models.
- Activity 2.2: Build the ML inference module (app/ml.py) that loads the trained model and scaler.
- Activity 2.3: Implement the /api/predict route with input validation and database logging.

Milestone 3: Frontend Development

- Activity 3.1: Build responsive templates (home, predict, history) with a blue/green design system.
- Activity 3.2: Implement AJAX form submission and dynamic result rendering in JavaScript.

Milestone 4: Deployment

- Activity 4.1: Push the project to GitHub.
- Activity 4.2: Deploy on Render as a Web Service with an automated build/train step.
- Activity 4.3: Resolve deployment issues (Python version pinning, entry-point naming conflict).

Milestone 5: Conclusion & Future Enhancements

Milestone 1: Data Preparation and Model Selection

This milestone focuses on preparing a small, imbalanced weather dataset for reliable model training and selecting the classifier that best balances precision and recall.

Activity 1.1: Load the Dataset

The dataset (data/flood_dataset.xlsx) contains 115 records with Temp, Humidity, Cloud Cover, ANNUAL rainfall, four seasonal rainfall splits (Jan-Feb, Mar-May, Jun-Sep, Oct-Dec), avgjune, sub, and the binary flood target (99 no-flood vs 16 flood records).

Activity 1.2: Exploratory Data Analysis

train.py automatically saves a class-distribution bar chart and a feature-correlation heatmap to static/img/, which are displayed on the FloodGuard home page as live model insights.

Activity 1.3: Preprocessing

- Missing values are median-imputed per feature.
- Outliers are capped using the IQR method (clipped, not dropped, to preserve the small minority class).
- Features are scaled with StandardScaler before training.

Activity 1.4: Model Training and Comparison

Four classifiers are trained with class-balancing (class_weight='balanced' / scale_pos_weight) to counter the 99:16 class imbalance, then compared on Accuracy, Precision, Recall, and F1-score. The model is selected by F1-score rather than raw accuracy, and Random Forest is preferred when it lands within 5 points of the top score, since it produces smoother, more realistic probability outputs than a single Decision Tree.

Milestone 2: Core Backend Development

Activity 2.1: Flask App Factory & Models

`create_app()` in `app/__init__.py` builds the Flask app, initializes the SQLAlchemy database, ensures a default guest user exists, and syncs the `ml_models` table with whatever model `train.py` most recently produced.

Example — `app/__init__.py` excerpt:

```
def create_app():
    app = Flask(__name__)
    db.init_app(app)
    with app.app_context():
        db.create_all()
        models.ensure_default_user()
        models.sync_ml_model_registry()
    from app.routes import main_bp
    app.register_blueprint(main_bp)
    return app
```

Activity 2.2: ML Inference Module

`app/ml.py` loads `flood_model.joblib` and `scaler.joblib` once at import time and exposes `predict_flood(feature_dict)`, which scales the input, runs the model, and returns the prediction, flood/no-flood probabilities, a risk level, and the model name used.

Activity 2.3: Prediction API Route

Example — `app/routes.py` excerpt:

```
@main_bp.route("/api/predict", methods=["POST"])
def api_predict():
    data = request.get_json(silent=True)
    numeric_data = {k: float(data[k]) for k in REQUIRED_FIELDS}
    result = ml.predict_flood(numeric_data)
    # ... persist WeatherData + PredictionResult, return jsonify(result)
```

Milestone 3: Frontend Development

The frontend uses a blue/green “environment” design system defined via CSS custom properties, Inter for UI text and IBM Plex Mono for numeric/data readouts. The homepage shows live model accuracy and auto-generated insight charts; the /predict page submits weather data via the Fetch API and renders the result (risk badge + animated probability bars) without a page reload; the /history page lists the most recent predictions from the database.

Activity 3.1: Responsive Templates

- base.html: shared navbar (with mobile hamburger toggle) and footer.
- home.html: hero section, feature grid, live model-insight charts, call-to-action.
- predict.html: two-column layout — input form and dynamic result card.
- history.html: scrollable table of past predictions with color-coded risk pills.

Activity 3.2: Dynamic Result Rendering

static/js/script.js attaches a submit handler to the prediction form that performs client-side validation, POSTs JSON to /api/predict, shows a loading spinner, and then animates the probability bars to their final widths using a short CSS transition.

Milestone 4: Deployment

Activity 4.1: Push to GitHub

Repository: <https://github.com/BSG8710/flood-prediction-system>

Activity 4.2: Deploy on Render

- Build Command: `pip install -r requirements.txt && python train.py` (retrains the model fresh on every deploy)
- Start Command: `gunicorn main:app`
- Environment Variable: `PYTHON_VERSION = 3.12.7` (pins the Python version so pip finds pre-built wheels instead of compiling from source)

Activity 4.3: Deployment Issues Resolved

Issue	Root Cause	Fix
Build stuck compiling pandas/numpy from source	Render defaulted to a newer Python version with no pre-built wheel for the pinned package versions	Added <code>PYTHON_VERSION=3.12.7</code> environment variable
<code>gunicorn: AttributeError: module 'app' has no attribute 'app'</code>	Naming clash between <code>app.py</code> (entry file) and the <code>app/</code> package folder	Renamed the entry file to <code>main.py</code> and updated the start command to <code>gunicorn main:app</code>
<code>app/__init__.py</code> <code>IndentationError</code> / <code>SyntaxError</code> after manual GitHub edits	Partial copy-paste left old and new code merged on the same line	Fully replaced the file content in one clean paste and re-verified the file ended correctly

Live deployed URL: <https://floodguard-ytqr.onrender.com>

Exploring the Website's Web Pages

Home Page

The landing page introduces FloodGuard, shows live model accuracy and prediction-count stats, a feature grid, and the auto-generated model-comparison / confusion-matrix / correlation charts.

Prediction Page

A two-column layout: a form on the left collects 10 weather values, and a result card on the right displays a risk badge, a short message, and two animated probability bars (flood vs no-flood) once a prediction is returned.

History Page

A responsive table of the most recent predictions, each row showing the submitted weather values, the model used, the result (Safe / Flood Risk pill), and the flood probability.

Conclusion

FloodGuard is a complete, deployed machine-learning system that turns raw weather measurements into an actionable flood-risk assessment. It demonstrates the full pipeline expected of a production ML product: careful handling of an imbalanced dataset, comparison of multiple algorithms with a metric appropriate to the problem (F1-score over raw accuracy), a clean Flask backend with a properly normalized database schema, a responsive and accessible frontend, and a live cloud deployment on Render. Looking ahead, the project has clear expansion paths: migrating prediction history to a persistent PostgreSQL database, adding user authentication using the existing User table, incorporating live weather-API data instead of manual entry, and retraining periodically as more real flood events are recorded.